Software Requirements Specification (SRS)

Campus Ride-Share Matching

Version: 1.0

Authors: Khushi P, Keertika , Tanishkaa , Pavithra

Date: 27 October 2026

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for Campus Ride-Share Matching, a platform that helps students coordinate shared rides by matching ride offers and requests using pickup/drop-off points and route compatibility.

1.2 Scope

The platform allows students to create ride offers or requests, provide pickup and destination details, discover compatible rides, communicate through in-app messaging, and manage their ride posts. The system supports geospatial query handling and matching heuristics. Real-time vehicle tracking, payment processing, and external transport-provider integrations are outside the initial scope unless added later.

1.3 Audience

Students, project developers, testers, project evaluators, and future maintainers.

1.4 Definitions

Ride offer: a student offering available seats in a ride. Ride request: a student seeking a ride. Geospatial query: a location-based query. Route compatibility: the degree to which pickup and destination points fit a ride's route. Match: a candidate offer/request pair that satisfies configured matching rules.

2. Overall Description

2.1 Product Perspective

The product is a student-facing application with a user interface, application/service layer, geospatial data/query component, matching component, persistence layer, and in-app messaging/notification component.

2.2 Major Product Functions

1. Student registration and sign-in. 2. Create, view, edit, and cancel ride offers. 3. Create, view, edit, and cancel ride requests. 4. Store pickup and drop-off locations. 5. Search for rides by location and relevant ride details. 6. Rank compatible offers and requests using route-matching heuristics. 7. Display match details and status. 8. Send and receive in-app messages. 9. Notify users about relevant match or message activity. 10. Validate inputs and report errors.

2.3 User Roles and Characteristics

Student: creates offers or requests, searches for rides, reviews matches, and messages other students. Project administrator (if enabled): handles basic moderation and platform configuration.

2.4 Operating Environment

A modern web browser or supported mobile interface, application server, persistent database, and a location/geospatial query capability. Exact deployment platform and technology versions are to be confirmed by the team.

2.5 Constraints and Assumptions

Users provide reasonably accurate pickup and destination locations. Matching quality depends on the accuracy of location and route information. The system should expose only the student information needed to coordinate a ride. Campus authentication, map provider, and production hosting choices depend on project scope and available services.

3. External Interface Requirements

3.1 User Interface

Provide clear forms for creating offers and requests, location fields, search/filter controls, match results, ride details, message views, and understandable validation/error messages.

3.2 Software Interfaces

The application communicates with its backend services and database through defined internal interfaces. A maps/geocoding provider may be integrated if selected by the team; it is not assumed to be available in the current prototype.

3.3 Communications

Client-server communication should use HTTPS in deployment. API requests should validate input and return consistent success/error responses. **3.4 Data Interfaces**

Core data entities include Student/User, RideOffer, RideRequest, Location, Match, and Message/Notification.

4. System Features (Detailed)

4.1 Account Access

FR-01: The system shall allow a student to register or sign in using the configured authentication method. FR-02: The system shall prevent unauthenticated users from creating or modifying ride posts. FR-03: The system shall allow users to update basic profile information required for ride coordination.

4.2 Ride Offers

FR-04: The system shall allow a student to create a ride offer with pickup, destination, departure date/time, available seats, and optional notes. FR-05: The system shall allow the owner to edit or cancel an active offer. FR-06: The system shall display the details and current status of an offer.

4.3 Ride Requests

FR-07: The system shall allow a student to create a ride request with pickup, destination, desired date/time, and optional notes. FR-08: The system shall allow the owner to edit or cancel a request. FR-09: The system shall display the details and current status of a request.

4.4 Geospatial Search and Matching

FR-10: The system shall store pickup and drop-off location information in a consistent format. FR-11: The system shall retrieve candidate offers/requests within configured geographic bounds. FR-12: The system shall evaluate route compatibility using documented matching heuristics. FR-13: The system shall rank candidate matches according to the configured compatibility score. FR-14: The system shall show the relevant locations, time details, and match status to the user. FR-15: The system shall avoid presenting cancelled or expired posts as active matches.

4.5 Messaging and Notifications

FR-16: The system shall allow matched or interested students to exchange in-app messages, subject to project rules. FR-17: The system shall display new messages and relevant match-status updates. FR-18: The system shall validate message content and associate each message with its sender, recipient/conversation, and timestamp.

4.6 Errors and Administration

FR-19: The system shall validate required fields and reject invalid dates, seat counts, or missing locations. FR-20: The system shall display a useful error when a search or matching operation fails. FR-21: If administrative moderation is implemented, authorized administrators shall be able to review and remove reported content.

5. Non-Functional Requirements

5.1 Performance

NFR-01: For the agreed test dataset and normal load, common searches should return results within a target of 3 seconds.
The team should measure and confirm this target during testing.

5.2 Reliability

NFR-02: The application should handle failed requests gracefully and preserve saved ride posts and messages after successful database writes.

5.3 Security and Privacy

NFR-03: The system shall restrict modification of a ride post to its owner or an authorized administrator. NFR-04: The system shall protect authenticated routes and avoid exposing passwords, tokens, or private data in logs. NFR-05: Production traffic shall use HTTPS.

5.4 Usability and Compatibility

NFR-06: Forms and match results shall use clear labels and validation feedback. NFR-07: The interface should remain usable on common desktop and mobile screen sizes.

5.5 Data Integrity

NFR-08: The system shall validate seat availability, required location fields, and post status before confirming or displaying a match.

6. Quality Attributes and Acceptance Tests

Acceptance criteria

A student can create an offer and a request; location-based search returns candidate posts; matching logic ranks compatible routes; cancelled posts are excluded; messages are associated with the correct conversation; invalid inputs are rejected; and unauthorized changes are blocked.

7. UML Use-Case Diagram

Actors and use cases

Primary actor: Student. Use cases: Register/Sign In, Create Ride Offer, Create Ride Request, Search Rides, View Match Suggestions, Manage Own Posts, Send/Read Messages, Receive Notifications. Optional actor: Administrator — Review Reports/Remove Inappropriate Posts. Insert the team's UML use-case diagram in this section before submission.

8. Requirements Traceability Matrix (RTM)

Example traceability

FR-04 Create ride offer → Offer creation test. FR-07 Create ride request → Request creation test. FR-11 Geospatial candidate search → Location search test. FR-12/FR-13 Route matching and ranking → Matching test. FR-16 Messaging → Message delivery/display test. NFR-01 Search response time → Performance test. NFR-03 Authorization → Access-control test.

9. Security Objectives and Requirements

Security objectives

SO-01: Protect student account and ride-related information from unauthorized access. SO-02: Preserve integrity of ride posts, match status, and messages.

Security requirements

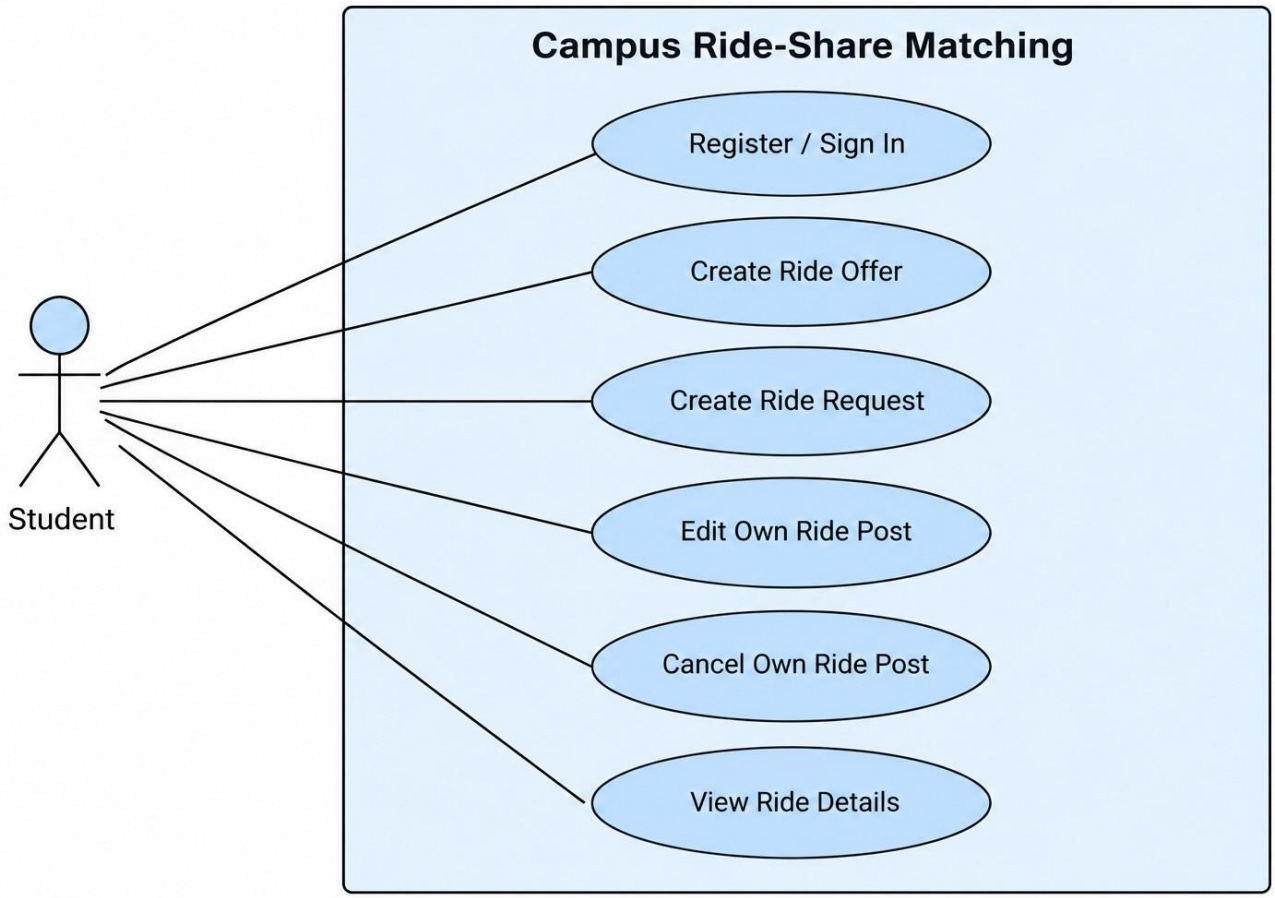
SR-01: Authenticated identity shall be checked before protected actions. SR-02: Users shall only edit or cancel their own posts unless granted an administrative role. SR-03: Secrets and credentials shall not be stored in source control. SR-04: Inputs shall be validated on the server. SR-05: Production client-server traffic shall use HTTPS.

10. Appendices

Glossary and references

Glossary: FR = Functional Requirement; NFR = Non-Functional Requirement; RTM = Requirements Traceability Matrix; UI = User Interface; API = Application Programming Interface. References: project brief provided by the team; selected implementation framework/database documentation; team-defined matching rules and test cases.

Section 7 · UML Use-Case Diagram 1



Section 7 · UML Use-Case Diagram 2

